Research Gap: Accelerator DMA vs. CPU Deadlines in TinyML SoCs

Oct 30, 2026 · @Abdelrahman Youssef

The gap: TinyML accelerator studies tune DMA for inference speed alone, while deadline-aware bus studies target FPGA SoCs with generic accelerators. This project measures the DMA's effect on a co-running CPU deadline, and the SystemC proof shows the fastest settings lose 33–46% of 16 kHz samples.

- **Research question.** On an MCU-class TinyML SoC, which accelerator DMA setting (bus mode, burst size) gives the shortest inference while a 16 kHz CPU task still meets its deadline, and does the answer change with the model and the CPU's workload?
- **Literature evidence.** Among 20 surveyed works, the deadline-aware ones use FPGA or DRAM SoCs, and the TinyML ones time inference only. MLPerf Tiny's own rules place only inference in its measurement window.
- **SystemC evidence.** Your RISC-V VP++ + STONNE platform runs 43 configurations. On an ideal bus the task never misses. On a shared bus, settings within 1.2% of each other in inference time range from 0% to 46% of samples missed. Meeting every deadline costs at most 1% of inference time when the CPU sleeps, and 24–57% when it is busy.
- **Whose work it extends.** Brilli et al. (DATE 2022) and Restuccia et al. (ACM TECS 2019, IEEE TC 2022), both on the instructor's venue list, with STONNE (IEEE CAL 2021) as the timing engine.
- **How to phrase it in the paper.** Say what the work does ("this work measures the effect of the accelerator's DMA settings on a co-running CPU deadline"), and let the matrix and the runs show that the question is open.

The gap, stated precisely

On a TinyML system-on-chip, the accelerator runs the network while the CPU keeps its time-critical work. In keyword spotting, the CPU must take one audio sample every 62.5 µs (16 kHz) while the accelerator runs DS-CNN on the previous window. The accelerator's DMA and the CPU share one bus. So every DMA setting that speeds up inference, such as the bus-ownership mode (block, cycle stealing, transparent) or the burst size, also sets how long the CPU can be held off the bus.

Two lines of published work approach this from opposite sides:

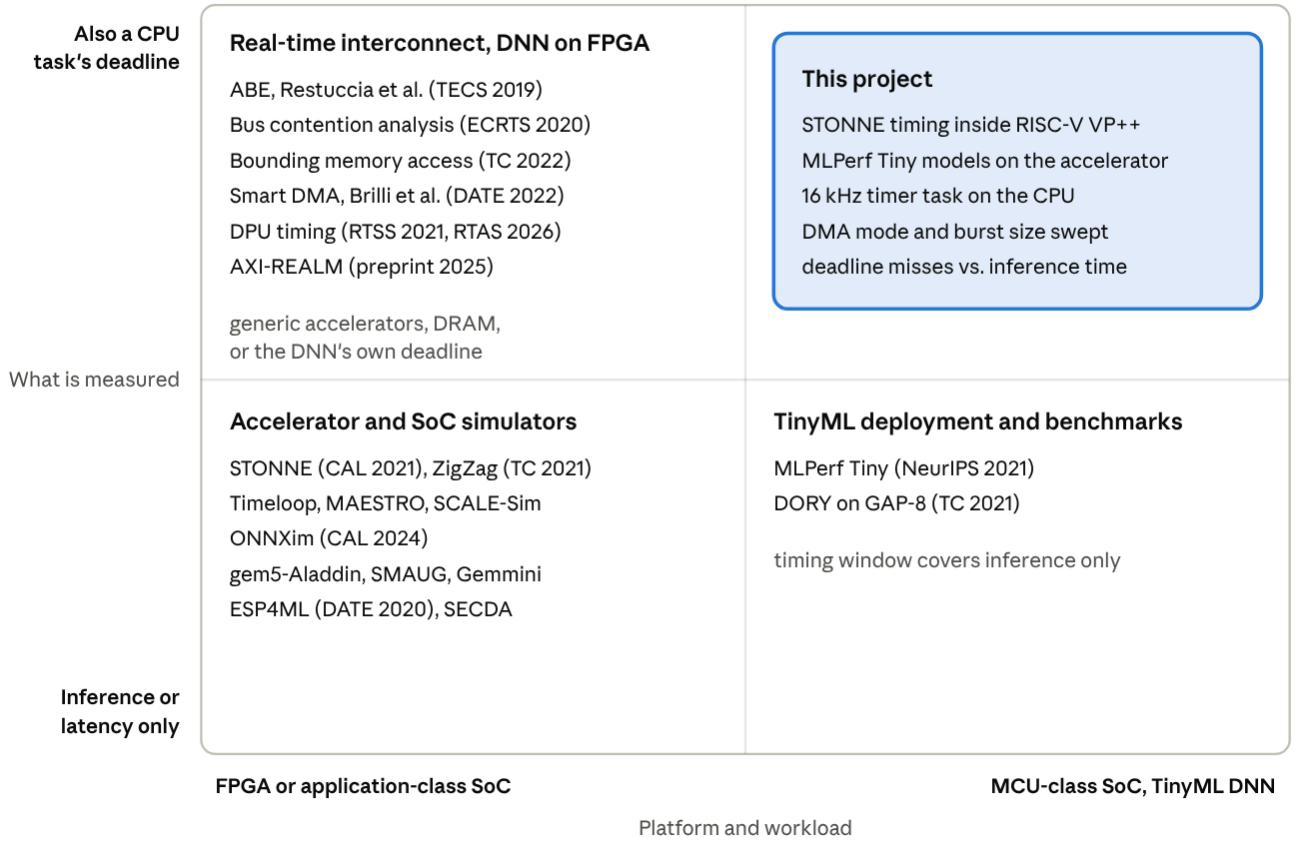
- **Accelerator design and TinyML deployment** (STONNE, ZigZag, DORY, MLPerf Tiny) tune the accelerator and its data movement for inference latency and energy, with the network running alone.

- **Real-time interconnect work** (Restuccia et al., Brilli et al., AXI-REALM) bounds how accelerator traffic delays other bus masters. It studies FPGA SoCs with DRAM, application-class cores, and generic accelerators, traffic generators, or the DNN's own deadline.

Research question. On an MCU-class TinyML SoC, which DMA configuration gives the shortest inference while a co-running CPU task still meets its deadline? How far is it from the configuration an inference-only evaluation picks, and does the answer change with the model and the task?

The gap is the meeting point of the two lines, not a missing topic in either one. Each line has the half of the problem the other lacks: the right workload and platform on one side, deadline-aware bus analysis on the other.

This project sits where deadline-aware bus studies meet TinyML SoCs



20 surveyed works placed by platform and by what they measure (detail in Evidence 1)

Read it by quadrant. The top-left works have deadlines but not TinyML SoCs, and the bottom-right works have TinyML SoCs but not deadlines. This project fills the top-right.

Evidence 1: what the surveyed literature measures

None of the 20 surveyed works places all six dimensions in one evaluation, and column E is where the two lines split. The TinyML and accelerator works have the right workload and platform but measure inference alone. The real-time works measure deadlines and bus interference, but on FPGA or DRAM SoCs and for generic accelerators or the DNN's own deadline.

Columns:

- **A**: DNN-accelerator timing, with layer-dependent traffic.
- **B**: MCU-class CPU and driver in the loop.
- **C**: CPU–DMA contention on a shared bus.
- **D**: DMA bus-ownership mode or burst size treated as a design parameter.
- **E**: deadline of a co-running CPU task measured.
- **F**: TinyML (MLPerf Tiny-class) workload.

Marks: ● central to the evaluation · ◐ present in another form (see the note) · — not part of the reported evaluation.

Work	Venue	A	B	C	D	E	F	Note
STONNE (Muñoz-Martínez et al.)	IEEE CAL 2021	●	—	—	—	—	—	Accelerator alone, operands assumed on chip
ZigZag (Mei et al.)	IEEE TC 2021	●	—	—	—	—	—	Analytical mapping and architecture search
Timeloop / MAESTRO / SCALE-Sim	ISPASS 2019, MICRO 2019, ISPASS 2020	●	—	—	—	—	—	Accelerator-only cost models
ONNXim (Ham et al.)	IEEE CAL 2024	●	—	◐	—	—	—	Multi-core NPU with NoC and DRAM; no host CPU task
gem5-Aladdin (Shao et al.)	MICRO 2016	◐	●	●	◐	—	—	SoC interface (DMA vs. coherent cache) changes the best accelerator design
SMAUG (Xi et al.)	ACM TACO 2020	●	●	◐	◐	◐	—	DMA vs. ACP; a 33 ms deadline for the inference pipeline itself
Gemmini (Genc et al.)	DAC 2021	●	●	◐	—	—	—	Shared TLB and L2 between CPU and accelerator

Work	Venue	A	B	C	D	E	F	Note
ESP4ML (Giri et al.)	DATE 2020	●	●	◐	—	—	◐	NoC SoC, accelerator pipelines
SECDA (Haris et al.)	SBAC-PAD 2021	●	●	—	◐	—	◐	SystemC + FPGA co-design for edge inference
DORY (Burrello et al.)	IEEE TC 2021	●	●	◐	◐	—	◐	Tiled, double-buffered DMA on GAP-8
MLPerf Tiny (Banbury et al.)	NeurIPS 2021	—	●	—	—	—	●	Timing window covers inference only
ABE (Restuccia et al.)	ACM TECS 2019	—	◐	●	●	◐	—	Burst-size heterogeneity makes round-robin AXI unfair
Bus contention analysis (Restuccia et al.)	ECRTS 2020	—	—	●	●	●	—	Response-time bounds for FPGA accelerators
Bounding memory access (Restuccia et al.)	IEEE TC 2022	—	—	●	◐	●	—	Schedulability of multi-accelerator AXI systems
CMRI / Smart DMA (Brilli et al.)	DATE 2022	◐	◐	●	●	◐	—	DMA splits transfers to protect CPU tasks on Zynq and Versal
DNN on FPGA SoC (Restuccia & Biondi)	IEEE RTSS 2021	●	—	●	—	◐	—	Response-time bounds of the DPU's own inference
Multi-core DPUs (Aromolo et al.)	IEEE RTAS 2026	●	—	●	◐	◐	—	Bandwidth budgets and preemptive DNN scheduling
AXI-REALM (Benz et al.)	arXiv 2025 (preprint)	—	—	●	●	◐	—	Burst fragmentation and budgets in AXI4
This project	COE 407/661	●	●	●	●	●	●	STONNE timing inside RISC-V VP++, 16 kHz CPU task

MLPerf Tiny makes the inference-only window explicit: it "specifically targets model inference and does not include pre- or post-processing in the measurement window" (Banbury et al.). A TinyML accelerator ranked by this benchmark, or by any column-A tool, is ranked with no other task on the CPU.

Evidence 2: the closest work, and what this project builds on it

This project carries a question already answered for FPGA SoCs into the TinyML design space: how far an accelerator's DMA bursts delay the CPU. It builds on two papers from the instructor's venue list, and STONNE (IEEE CAL 2021) remains the timing engine.

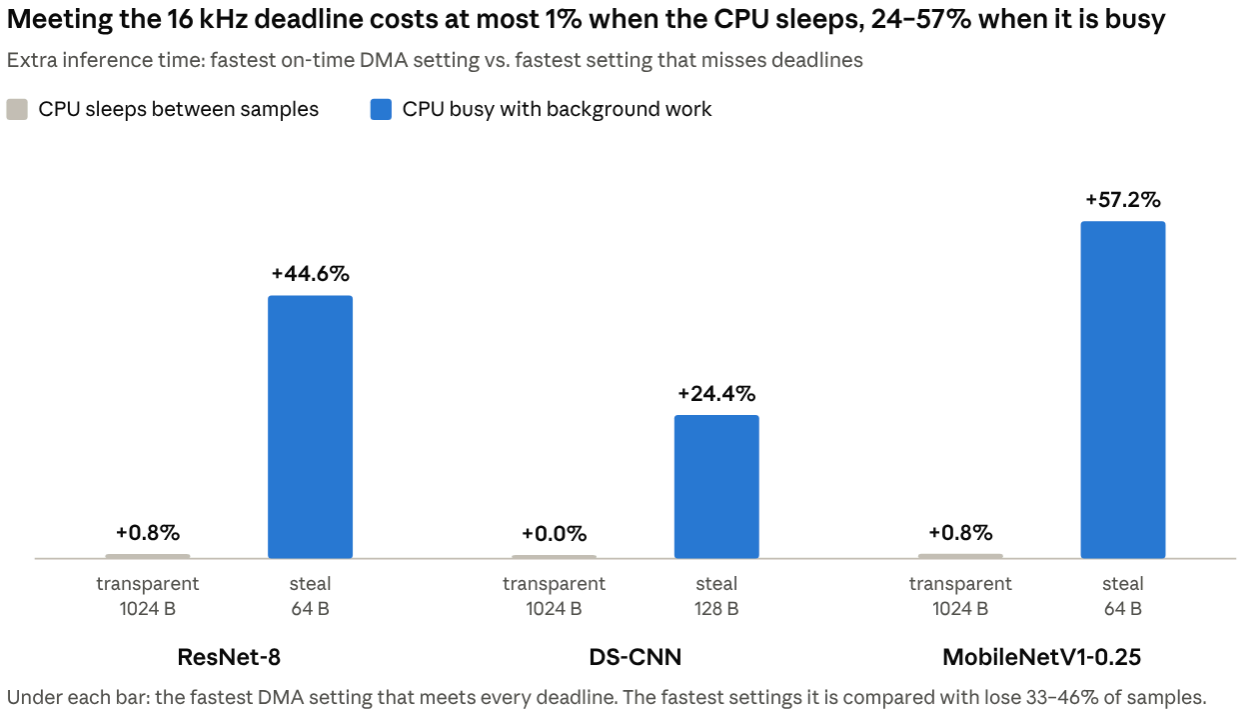
Closest work	What it establishes	What this project adds
Brilli et al., DATE 2022	FPGA accelerator clusters slow CPU tasks by up to 16× on Zynq UltraScale+ and Versal; a Smart DMA that splits transfers and inserts idle cycles keeps the slowdown under 10%.	The same lever (how the DMA splits and holds the bus) applied to a DNN accelerator on a single-bus MCU, scored by a CPU task's deadline instead of its slowdown, with traffic that changes layer by layer.
Restuccia et al., ACM TECS 2019	Round-robin AXI arbitration becomes unfair when masters use different burst sizes; in some configurations one master can arbitrarily reduce the bandwidth the others get.	Measures that effect for the DMA burst sizes a DNN accelerator is tuned to (16–1024 B), and shows which burst sizes keep a 16 kHz task on time.
Restuccia et al., IEEE TC 2022; ECRTS 2020	Response-time bounds for hardware accelerators that share an AXI interconnect and DRAM.	A simulation-based measurement for an MCU-class SoC, where the CPU is the victim and the DNN accelerator is the source of interference.
Restuccia & Biondi, RTSS 2021; Aromolo et al., RTAS 2026	Time-predictable DNN inference on Xilinx DPUs: bounds on the DNN's own response time under memory interference.	The other direction: the DNN accelerator's effect on a CPU task, for TinyML models.
MLPerf Tiny, NeurIPS 2021 and DORY, IEEE TC 2021	The TinyML workload, the MCU platform class, and tiled, double-buffered DMA tuned for inference latency.	The same models and DMA design choices, with a co-running task added to the measurement.

In short, this project measures the effect of the DNN accelerator's DMA settings on the CPU's deadlines, for TinyML models on an MCU-class SoC, inside one SystemC virtual prototype.

Evidence 3: the SystemC proof

The experiment reproduces the gap in your own platform. The fastest DMA settings lose 33–46% of the 16 kHz samples, a model without bus contention shows no misses at all, and the setting that fixes it depends on what the CPU is doing.

Setup. RISC-V VP++ (RV32IM, 100 MHz) runs the step-3b accelerator: MAERI, 256 multipliers, STONNE timing, conv partition, completion interrupt. A CLINT timer releases a task every 62 µs (16.1 kHz). The task reads one sample, runs a 16-tap FIR filter, and must finish before the next release. Meanwhile all 48 MLPerf Tiny layers run on the accelerator. The sweep covers 43 runs: 4 bus modes × burst sizes 16–1024 B × 3 scenarios (16-tap task with the CPU sleeping, 64-tap task with the CPU sleeping, 16-tap task with the CPU busy with background work). Code: `sw/bench_rt.c`, `scripts/gap_sweep.py`.



results/gap_summary.csv · 43 runs · DMA mode (one piece), 16-tap task, 62 µs period

What the runs show.

1. **Ideal bus: no misses anywhere.** On the ideal bus that accelerator-only tools and steps 1–4 assume, the task never misses in any of the 8 ideal-bus runs, for any model. Its worst response is 739 cycles against a 6,200-cycle deadline. The problem is invisible to that model.

2. **Block-mode DMA misses at every burst size.** 17–93% of samples are late or lost, depending on model and burst size, and the longest response is 124,840 cycles (1.25 ms, 20 sample periods).
3. **Steal mode is on time only with small bursts.** At 64 B the worst response is about 2,500 cycles. From 128 B up, 15–46% of samples miss with the CPU asleep. The CPU's worst response grows with the burst size, to about 17,000 cycles at 1024 B, because each CPU bus access can wait for a whole burst.
4. **Inference time cannot tell the safe setting from the unsafe ones.** At 1024 B the three bus modes finish within 1.2% of each other, yet their misses range from 0% to 46% (table below).
5. **The right setting depends on the CPU's workload.** With the CPU asleep between samples, transparent DMA is on time at almost no cost. With the CPU busy, transparent DMA is still on time but 47–61% slower, and the best on-time choice is steal mode with 64–128 B bursts, at +24% to +57%.

ResNet-8 and MobileNet, CPU asleep, 1024-B bursts:

Bus mode	ResNet-8 time	ResNet-8 samples late or lost	MobileNet time	MobileNet samples late or lost
block	2.092 ms	45.7%	4.369 ms	41.2%
steal	2.111 ms	34.3%	4.421 ms	43.9%
transparent	2.108 ms	0%	4.405 ms	0%

An inference-only evaluation would rank these three as equal and could pick any of them. Only the deadline metric separates them.

Why it matters

A late or lost sample is a functional error, not only a slower system.

- **Keyword spotting.** A lost audio sample is a hole in the window that the next DS-CNN inference reads. MLPerf Tiny leaves feature extraction out of its measurement and feeds the model prepared inputs, so that loss never shows in its scores.
- **Other TinyML uses.** Motor control, IMU sensor fusion and radio stacks have the same shape: a periodic CPU deadline beside an accelerator that bursts on the bus.
- **Where it is found.** With inference-only numbers, the DMA setting is chosen first and the conflict shows up at system integration on silicon. A SystemC virtual prototype finds it before any hardware exists, which is the course's case for system-level modelling.

Objections the professor may raise, and the answers

Objection	Answer
"Textbooks already say burst-mode DMA stalls the CPU."	They give the mechanism, not the design answer. Which mode and burst size a DNN accelerator needs for a 16 kHz task, what that costs in inference time, and how it changes with the model and the CPU's workload are numbers no textbook gives. The experiment produces them.
"Restuccia et al. and Brilli et al. already did this."	They did it for FPGA SoCs with DRAM and AXI, with generic accelerators or traffic generators. This project uses a single-bus MCU, DNN-accelerator traffic that changes layer by layer, a deadline as the metric, and MLPerf Tiny models. It extends their question; it does not repeat it.
"Real MCUs have bus matrices, separate SRAM banks or CPU-priority arbiters."	Some do. The shared, first-come-first-served bus is the base case. Each of those features is a candidate fix, and comparing their cost is the design exploration this gap opens.
"It is only a simulation."	System-level modelling is the course's method, and the virtual prototype is deterministic. The claims are comparative (which setting misses, and by how much), not absolute times on a chip.
"Your CPU is slow, so the task is artificially long."	Without contention the task takes about 360 cycles, under 6% of its 6,200-cycle period. The misses come from how long the DMA holds the bus, not from the task's length.
"Raise the task's priority in software."	Priority decides which code runs next. It cannot take the bus back from a DMA transfer that already holds it.

Making the literature evidence airtight

The matrix above comes from a targeted web search plus the papers already in the term paper. Before the professor meeting, repeat it as a documented database search, so the "20 works" claim rests on a protocol rather than on recall.

1. **Databases.** Search IEEE Xplore, ACM Digital Library, Scopus and dblp, years 2016–2026.
2. **Search strings**, each run separately:
   a. ("DNN accelerator" OR "neural processing unit" OR NPU) AND (DMA OR "bus contention" OR interference) AND ("real-time" OR deadline OR "interrupt latency")
   b. (TinyML OR microcontroller OR MCU) AND accelerator AND (interference OR "shared bus" OR contention)
   c. ("cycle stealing" OR "burst size" OR "DMA mode") AND (accelerator OR "neural network")
   d. "MLPerf Tiny" AND (concurrent OR "co-running" OR interference OR real-time)
3. **Snowballing.** Read the forward citations ("cited by") of Brilli et al. DATE 2022 and Restuccia et al. IEEE TC 2022. A closer match, if one exists, most likely cites one of these two.
4. **Screening rule.** A work closes the gap only if it has all three of: a DNN accelerator, a CPU task with a deadline, and the DMA or bus setting varied.
5. **Record.** For each string, write down hits, the number screened by title and abstract, and the number kept. Add each kept paper as a row of the matrix. One small table of these counts in the term paper is the proof the professor asks for.

If the search finds a paper that already closes the gap, the project still stands. It becomes a reproduction in SystemC on MLPerf Tiny, and the framing changes from "measures" to "reproduces and extends".

Limits of the evidence

- **Literature.** The matrix comes from a targeted search, not a systematic review; the protocol above closes that.
- **Bus model.** The arbiter is analytical: first come, first served, with no split transactions or priorities. A priority arbiter that favours the CPU would shorten the CPU's waits, and testing one is a natural next step.
- **CPU timing.** The tiny32 timing model is slow and cacheless, so the task takes about 360 cycles even with no contention. A faster core shortens the task, but not the time a DMA tenure holds the bus, so block-mode misses remain.
- **One task, bare metal.** The setup has one periodic task and no RTOS. Layers run back to back on fresh data, without pooling or requantization.
- **Timer resolution.** mtime counts whole microseconds, so the period is 62 µs (16.1 kHz) instead of 62.5 µs.

- **Release phase.** Miss percentages depend on where each 62 µs release falls relative to a transfer. A rebuilt program moved them by a few points (for example, 39.5% to 45.7%), but the pattern did not change. Quote ranges, not single values.
- **Tool reliability.** This work found and fixed a STONNE bug: `MSwitch` left its link pointers uninitialized, which could abort a run depending on heap layout. The fix is `patches/stonne-mswitch-init.patch`, and `scripts/regress_all.sh` re-checks every earlier result against `reference_results/`.

References

★ = on the instructor's venue list (JSA, IEEE CAL, IEEE TC, ACM TECS, DATE). Links are the pages opened for this note; the rest come from the term paper's bibliography.

1. ★ F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN inference accelerators," *IEEE CAL*, 20(2), 2021.
2. ★ F. Restuccia, M. Pagani, A. Biondi, M. Marinoni, G. Buttazzo, "Is your bus arbiter really fair? Restoring fairness in AXI interconnects for FPGA SoCs," *ACM TECS*, 18(5s), 2019. record
3. F. Restuccia, M. Pagani, A. Biondi, M. Marinoni, G. Buttazzo, "Modeling and analysis of bus contention for hardware accelerators in FPGA SoCs," *ECRTS*, LIPIcs 165, 2020. record
4. ★ F. Restuccia, M. Pagani, A. Biondi, M. Marinoni, G. Buttazzo, "Bounding memory access times in multi-accelerator architectures on FPGA SoCs," *IEEE TC*, 72(1), 154–167, 2022. record
5. ★ G. Brilli, A. Capotondi, P. Burgio, A. Marongiu, "Understanding and mitigating memory interference in FPGA-based HeSoCs," *DATE*, 1335–1340, 2022. paper
6. F. Restuccia, A. Biondi, "Time-predictable acceleration of deep neural networks on FPGA SoC platforms," *IEEE RTSS*, 2021. paper
7. F. Aromolo, N. Salamini, J. Del Granchio, A. Biondi, M. Marinoni, G. Buttazzo, "Time-predictable acceleration of deep neural networks on FPGA SoCs with multi-core DPUs," *IEEE RTAS*, 2026. paper
8. T. Benz et al., "AXI-REALM: Safe, modular and lightweight traffic monitoring and regulation for heterogeneous mixed-criticality systems," arXiv:2501.10161, 2025 (preprint). paper
9. C. Banbury et al., "MLPerf Tiny benchmark," *NeurIPS Datasets and Benchmarks*, 2021. paper
10. ★ A. Burrello et al., "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE TC*, 70(8), 2021. paper

11. ★ L. Mei et al., "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE TC*, 70(8), 2021.
12. A. Parashar et al., "Timeloop," *ISPASS* 2019; H. Kwon et al., "MAESTRO," *MICRO* 2019; A. Samajdar et al., "SCALE-Sim," *ISPASS* 2020.
13. ★ H. Ham et al., "ONNXim: A fast, cycle-level multi-core NPU simulator," *IEEE CAL*, 2024.
14. Y. S. Shao et al., "Co-designing accelerators and SoC interfaces using gem5-Aladdin," *MICRO*, 2016.
15. S. L. Xi et al., "SMAUG: End-to-end full-stack simulation infrastructure for deep learning workloads," *ACM TACO*, 18(1), 2020. paper
16. H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," *DAC*, 2021. paper
17. ★ D. Giri et al., "ESP4ML: Platform-based design of systems-on-chip for embedded machine learning," *DATE*, 2020.
18. J. Haris et al., "SECDA: Efficient hardware/software co-design of FPGA-based DNN accelerators for edge inference," *SBAC-PAD*, 2021.
19. ★ V. Herdt, D. Große, P. Pieper, R. Drechsler, "RISC-V based virtual prototype," *JSA*, 109, 2020; M. Schlägl, C. Hazott, D. Große, "RISC-V VP++," *OSDA*, 2024.